

OVDAS-Core

Sistema de Monitoreo Volcánico
Arquitectura de Microservicios Orquestados

Manual de Instalación y Ejecución

Versión

2.3.0

Fecha

10 de marzo de 2026

Autor

Joaquín Aracena

Stack

Python 3.11 · FastAPI · RabbitMQ · PostgreSQL · K3s

Índice

1. Descripción General del Sistema	4
1.1. Arquitectura	4
1.2. Servicios y Puertos	4
1.3. Pipeline de Procesamiento	5
2. Prerrequisitos	6
2.1. Hardware Mínimo Recomendado	6
2.2. Software Requerido	6
3. Instalación — Opción A: Docker Compose (Desarrollo Local)	7
3.1. Clonar el Repositorio	7
3.2. Estructura de Directorios	7
3.3. Levantar el Stack Completo	7
3.4. Verificar que los Servicios Estén Activos	8
3.5. Escalar Workers	8
3.6. Detener el Stack	9
4. Instalación — Opción B: Kubernetes con K3s (Producción)	10
4.1. Instalar K3s y Docker	10
4.2. Construir e Importar Imágenes	10
4.3. Desplegar los Manifiestos Kubernetes	10
4.4. Verificar el Despliegue	11
5. Variables de Entorno	12
6. Inicialización de la Base de Datos	13
7. Ejecución y Flujo de Trabajo	14
7.1. Prueba de Extremo a Extremo (test-flow.sh)	14
7.2. Ingesta Manual vía API REST	14
7.3. Consulta de Resultados	15
7.4. Archivos MiniSEED — Volumen Compartido	15
8. Descripción de Servicios	16
8.1. Core Orchestrator	16
8.2. Worker Detección (ADOVE Pipeline)	16
8.3. Worker Picking (Castilla Picker)	16
8.4. Worker Localización (Hyposat)	17
8.5. WWS Poller	17
8.6. API Gateway (API Composition)	17
9. Mensajería RabbitMQ	18
10. Base de Datos PostgreSQL — Esquemas	20
11. Monitoreo y Observabilidad	22
12. Escalado y Alta Disponibilidad (K3s)	23
13. Resolución de Problemas	24

14. Módulos Futuros	26
15. Referencias	27

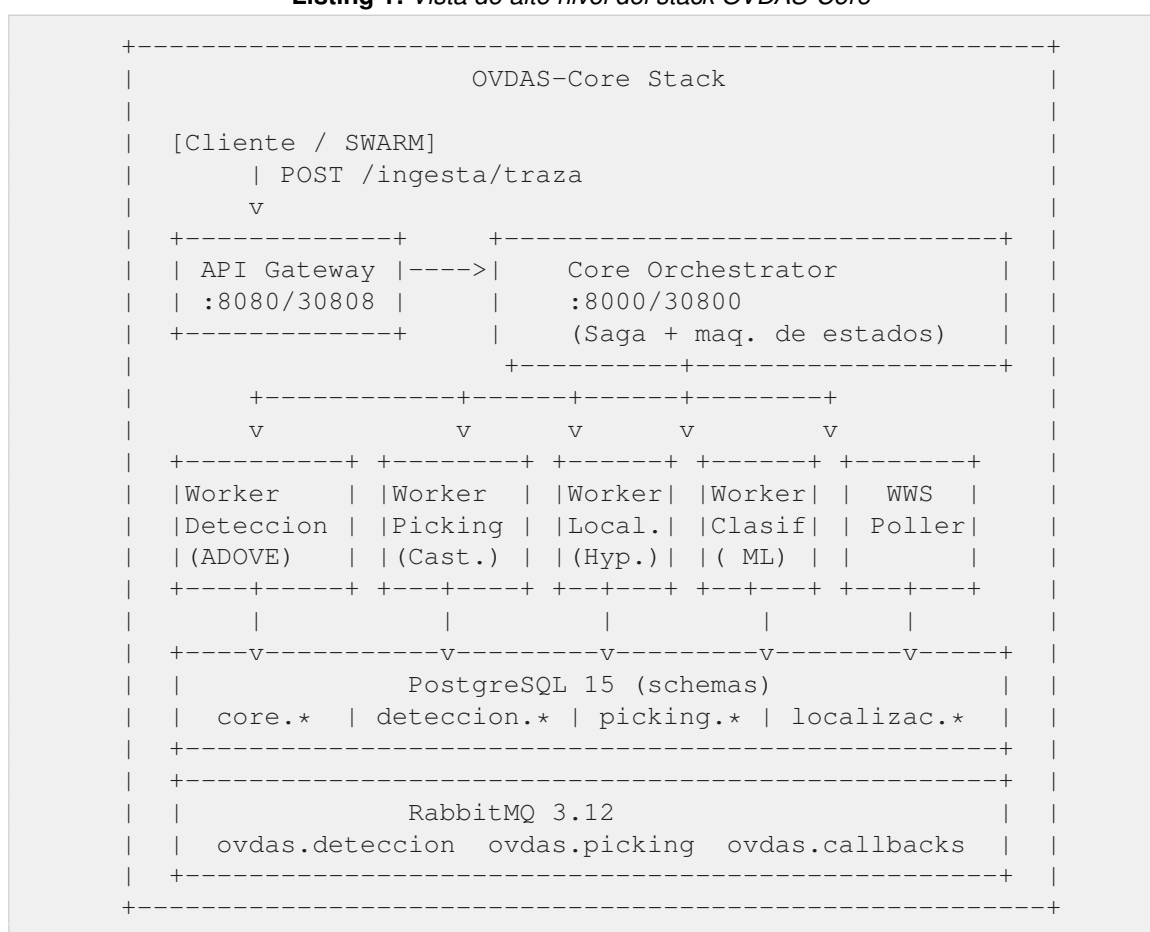
1. Descripción General del Sistema

OVDAS-Core es la reimplementación arquitectónica del sistema de monitoreo volcánico del Observatorio Volcanológico de los Andes del Sur (OVDAS-SENAPRED). El sistema transforma el monolito legacy (MySQL + scripts Python acoplados) en una arquitectura de microservicios orquestados, desplegable en entornos on-premise mediante Kubernetes ligero (K3s).

1.1 Arquitectura

El sistema implementa el patrón **Saga de Orquestación**: el servicio *Core* actúa como orquestador central que coordina los workers a través de colas RabbitMQ. Cada worker es independiente, posee su propio esquema de base de datos (Database-per-Service) y comunica sus resultados mediante callbacks asíncronos.

Listing 1: Vista de alto nivel del stack OVDAS-Core



1.2 Servicios y Puertos

Servicio	Puerto Docker	Puerto K3s	Descripción
core	:8000	30800	Orquestador Saga + API REST
api-gateway	:8080	30808	Composición de resultados
worker-deteccion	:8001	—	ADOVE + VGG16 (CPU)
worker-picking	—	—	Castilla Picker P/S

Servicio	Puerto Docker	Puerto K3s	Descripción
worker-localizacion	—	—	Hyposat (hipocentro)
wws-poller	—	—	Descarga MiniSEED desde Winston
rabbitmq (mgmt)	:15672	31672	Broker AMQP + UI web
postgres	:5432	—	Base de datos central
redis	:6379	—	Cache en memoria (uso futuro)

1.3 Pipeline de Procesamiento

Cada traza sísmica ingresada recorre los siguientes estados secuenciales:

```
INGRESADO -> DETECTANDO -> PICKING ->
LOCALIZANDO -> COMPLETADO

\ (error) -> ERROR
```

Estado	Descripción
INGRESADO	Traza recibida por el Core, <code>evento_id</code> asignado
DETECTANDO	Worker-detección procesando pipeline ADOVE Y Clasificación ML final
PICKING	Worker-picking calculando tiempos P y S
LOCALIZANDO	Worker-localización ejecutando Hyposat
COMPLETADO	Pipeline finalizado exitosamente
ERROR	Fallo en algún worker; mensaje de error almacenado

2. Prerrequisitos

2.1 Hardware Mínimo Recomendado

Componente	Mínimo	Recomendado
CPU	4 cores (x86_64)	8+ cores para worker-detección con VGG16
RAM	8 GB	16 GB con múltiples workers en paralelo
Disco	20 GB libres	SSD recomendado para PostgreSQL y datos Mini-SEED
Red	LAN interna	Conectividad al servidor Winston Wave Server (WWS)

2.2 Software Requerido

Para Docker Compose (desarrollo local)

Herramienta	Versión	Notas
Docker Engine	≥ 24.0	https://docs.docker.com/engine/install/
Docker Compose	≥ 2.20	Plugin incluido en Docker Desktop o por separado
Git	≥ 2.x	Para clonar el repositorio
curl / jq	cualquier	Para pruebas manuales desde terminal

Para K3s (producción on-premise)

Herramienta	Versión	Notas
Sistema Operativo	Fedora 38+, RHEL 9, Ubuntu 22.04	Probado en Fedora 43
Docker Engine	≥ 24.0	Para construir imágenes localmente
K3s	≥ 1.28	Instalado por <code>setup-k3s.sh</code>
kubectrl	con K3s	Configurado automáticamente
sudo / root	requerido	Solo para instalación de K3s y Docker

NOTA

El worker-detección descarga automáticamente PyTorch CPU (≈800 MB) durante el primer `docker build`. Asegúrese de tener conexión a Internet durante la primera construcción de imágenes.

3. Instalación — Opción A: Docker Compose (Desarrollo Local)

Docker Compose es el método recomendado para el desarrollo local y pruebas. Levanta todos los servicios en contenedores con redes y volúmenes gestionados automáticamente.

3.1 Clonar el Repositorio

```
# Clonar el repositorio (no hay remoto aun)
git clone <URL_REPOSITORIO> ovdas-core
cd ovdas-core/ovdas-core
```

NOTA

Si ya tienes el directorio descargado, simplemente navega a **ovdas-core/** (el directorio que contiene `docker-compose.yml`).

3.2 Estructura de Directorios

```
ovdas-core/
|-- docker-compose.yml          # Orquestacion local
|-- core/                      # Core Orchestrator (FastAPI + Saga)
|-- worker-deteccion/          # ADOVE pipeline + VGG16
|-- worker-picking/            # Castilla Picker P/S
|-- worker-localizacion/       # Hyposat
|-- wws-poller/                # Descarga MiniSEED desde Winston
|-- api-gateway/               # Composicion de resultados
|-- db/                        # Scripts SQL (init + migraciones)
|   |-- init.sql
|   |-- migrate_add_localizacion.sql
|   |-- migrate_estaciones_completo.sql
|   |-- migrate_estacion_varchar10.sql
|   `-- migrate_picking_snr.sql
|-- k8s/                       # Manifiestos Kubernetes
`-- scripts/
    |-- setup-k3s.sh
    |-- build-images.sh
    |-- deploy.sh
    `-- test-flow.sh
```

3.3 Levantar el Stack Completo

Desde el directorio **ovdas-core/** (donde reside `docker-compose.yml`):

```
# Primera ejecucion (descarga imagenes base + construye)
docker compose up --build

# Ejecuciones posteriores (sin reconstruir)
docker compose up

# En modo background (daemon)
docker compose up -d --build
```

Docker Compose levantará los servicios en el siguiente orden, respetando las dependencias y

los health checks:

1. **postgres** — Base de datos (espera healthcheck `pg_isready`)
2. **rabbitmq** — Broker de mensajes (espera healthcheck `rabbitmq-diagnostics`)
3. **redis** — Cache en memoria
4. **core** — Orquestador (espera postgres + rabbitmq)
5. **worker-deteccion** — Carga modelos VGG16 (~60–90 s)
6. **worker-picking** — Picker P/S
7. **worker-localizacion** — Hyposat
8. **wws-poller** — Descargador de trazas sísmicas
9. **api-gateway** — Agregador REST

NOTA

El worker-detección tarda entre 60 y 90 segundos en estar listo porque carga los pesos del modelo VGG16 y el clasificador OOD en memoria. El endpoint `GET /health` retornará `{"modelo_cargado": false}` durante ese tiempo.

3.4 Verificar que los Servicios Estén Activos

```
# Ver estado de contenedores
docker compose ps

# Logs de un servicio especifico
docker compose logs -f core
docker compose logs -f worker-deteccion

# Health checks manuales
curl http://localhost:8000/health      # Core
curl http://localhost:8080/health     # API Gateway
curl http://localhost:8001/health     # Worker Deteccion

# Interfaz RabbitMQ Management UI
# Abrir en navegador: http://localhost:15672
# Usuario: ovdas | Contraseña: ovdas
```

Servicio	URL	Notas
Core API (Swagger)	<code>http://localhost:8000/doc</code>	Docs interactiva
API Gateway (Swagger)	<code>http://localhost:8080/doc</code>	Docs interactiva
RabbitMQ Management	<code>http://localhost:15672</code>	<code>ovdas / ovdas</code>
Core healthcheck	<code>http://localhost:8000/hea</code>	JSON <code>{status: ok}</code>
Worker detección health	<code>http://localhost:8001/hea</code>	Estado del modelo

3.5 Escalar Workers

Para procesar múltiples trazas en paralelo se pueden levantar varias instancias:

```
# Escalar worker-deteccion a 3 instancias
```



```
docker compose up --scale worker-deteccion=3

# Escalar worker-picking a 2 instancias
docker compose up --scale worker-picking=2

# Reducir a 1 instancia
docker compose up --scale worker-deteccion=1
```

CONSEJO

RabbitMQ distribuye los mensajes entre todas las instancias del mismo worker en modo round-robin. Cada mensaje es consumido por exactamente una instancia.

3.6 Detener el Stack

```
# Detener sin eliminar contenedores ni volúmenes
docker compose stop

# Detener y eliminar contenedores (conserva volúmenes de datos)
docker compose down

# Eliminar TODO incluyendo volúmenes (!DESTRUYE LOS DATOS!)
docker compose down -v
```

! ADVERTENCIA

El comando `down -v` eliminará permanentemente la base de datos PostgreSQL y el almacenamiento de RabbitMQ. Úselo solo si desea un reinicio completo desde cero.

4. Instalación — Opción B: Kubernetes con K3s (Producción)

K3s es una distribución ligera de Kubernetes optimizada para entornos on-premise y edge computing. Es el modo de despliegue recomendado para producción en la infraestructura OVDAS.

4.1 Instalar K3s y Docker

El script `setup-k3s.sh` automatiza la instalación de Docker (vía `dnf` en Fedora/RHEL) y K3s, y configura `kubectl` para el usuario actual.

```
# Requiere privilegios de superusuario
sudo bash scripts/setup-k3s.sh

# El script realiza:
# 1. Instala Docker Engine via dnf (Fedora/RHEL)
# 2. Habilita e inicia el servicio Docker
# 3. Descarga e instala K3s (binary + systemd service)
# 4. Configura KUBECONFIG en ~/.kube/config del usuario actual
# 5. Espera a que el nodo este en estado Ready

# Verificar instalacion
kubectl get nodes
kubectl get namespaces
```

NOTA

En distribuciones basadas en Debian/Ubuntu, el script puede requerir modificación para reemplazar `dnf` por `apt-get`. Editar `scripts/setup-k3s.sh` antes de ejecutar.

4.2 Construir e Importar Imágenes

K3s utiliza su propio registro de imágenes (`containerd`) separado de Docker. El script `build-images.sh` construye las imágenes con Docker y las importa al runtime de K3s:

```
bash scripts/build-images.sh

# El script construye y etiqueta:
# ovdas/core:latest
# ovdas/worker-deteccion:latest
# ovdas/worker-picking:latest
# ovdas/api-gateway:latest

# Y los importa a K3s via:
# docker save <image> | sudo k3s ctr images import -

# Verificar imagenes importadas en K3s
sudo k3s ctr images ls | grep ovdas
```

4.3 Desplegar los Manifiestos Kubernetes

```
bash scripts/deploy.sh
```

```
# El script aplica los manifiestos en orden:
# 00-namespace.yaml      -> Namespace 'ovdas'
# 01-configmap-secrets.yaml -> ConfigMap + Secrets
# 02-postgres.yaml       -> StatefulSet PostgreSQL (PVC 10Gi)
# 03-rabbitmq.yaml       -> StatefulSet RabbitMQ (PVC 5Gi)
# 04-redis.yaml          -> Deployment Redis
# 05-core.yaml           -> Deployment Core + NodePort 30800
# 06-worker-deteccion.yaml -> Deployment Worker Deteccion
# 07-worker-picking.yaml -> Deployment Worker Picking
# 08-api-gateway.yaml    -> Deployment API Gateway + NodePort
#                          30808
# 09-hpa.yaml            -> HorizontalPodAutoscaler (worker-
#                          deteccion)
```

4.4 Verificar el Despliegue

```
# Ver todos los pods en el namespace ovdas
kubectl -n ovdas get pods

# Ver servicios y NodePorts asignados
kubectl -n ovdas get services

# Ver logs de un pod
kubectl -n ovdas logs -f deployment/core
kubectl -n ovdas logs -f deployment/worker-deteccion

# Healthchecks via NodePort (reemplazar IP_NODO por la IP del servidor
# )
curl http://IP_NODO:30800/health      # Core
curl http://IP_NODO:30808/health     # API Gateway
# RabbitMQ UI: http://IP_NODO:31672  (ovdas/ovdas)

# Ver HPA (escalado automatico)
kubectl -n ovdas get hpa
```

Servicio	NodePort	URL
Core API	30800	http://IP_NODO:30800/docs
API Gateway	30808	http://IP_NODO:30808/docs
RabbitMQ UI	31672	http://IP_NODO:31672 (ovdas/ovdas)

5. Variables de Entorno

Todas las variables se configuran en `docker-compose.yml` (desarrollo) o en el **ConfigMap/Secret de Kubernetes** (producción). Para modificarlas en K3s, editar `k8s/01-configmap-secrets.yml` y re-aplicar con:

```
kubectl apply -f k8s/01-configmap-secrets.yml
```

Variable	Servicio	Default	Descripción
RABBITMQ_URL	Todos	amqp://ovdas:ovdas@rabbitmq:5672/	URL de conexión AMQP
POSTGRES_DSN	Todos	host=postgres port=5432 dbname=ovdas ...	DSN PostgreSQL
REDIS_URL	Todos	redis://redis:	URL Redis (futuro)
DATA_DIR	Det./Pick./Poll.	/shared/data	Volumen MiniSEED compartido
OUT_DIR	Det./Pick.	/app/out	JSONs de auditoría
UMBRAL	Worker Detección	504	Umbral detección ADOVE
COMPONENTE	Worker Detección	Z	Componente sísmica (Z/N/E)
VOLCAN	Worker Detección	99	ID de volcán por defecto
MUESTRA_HZ	Worker Picking	100	Frecuencia de muestreo (Hz)
WWS_HOST	WWS Poller	—	Hostname servidor Winston
WWS_PORT	WWS Poller	16022	Puerto servidor WWS
POLL_INTERVAL	WWS Poller	30	Intervalo de sondeo (s)

6. Inicialización de la Base de Datos

La base de datos se inicializa automáticamente al levantar el stack. El script `db/init.sql` se ejecuta dentro del contenedor de PostgreSQL en el primer arranque (cuando el volumen está vacío).

Esquemas creados automáticamente

Schema	Tablas principales	Contenido
core	pipeline_eventos, volcanes, estaciones	Orquestación y catálogos
deteccion	resultados	Salidas del pipeline ADOVE
picking	resultados	Tiempos P/S del Castilla Picker
localizacion	resultados	Hipocentros de Hyposat
wws	descargas	Registro de MiniSEED descargados

Migraciones disponibles

Archivo	Descripción
<code>migrate_add_localizacion.sql</code>	Agrega schema <code>localizacion.resultados</code>
<code>migrate_estaciones_completo.s</code>	Carga catálogo completo de estaciones OVDAS
<code>migrate_estacion_varchar10.sq</code>	Amplía <code>estacion_id</code> a <code>VARCHAR(10)</code>
<code>migrate_picking_snr.sql</code>	Agrega columnas SNR y amplitud al schema picking

Ejecutar migraciones manualmente

```
# Identificar el contenedor de postgres
docker compose ps postgres

# Ejecutar migracion dentro del contenedor
docker compose exec postgres psql -U ovdas -d ovdas \
  -f /docker-entrypoint-initdb.d/migrate_add_localizacion.sql

# Verificar schemas y tablas
docker compose exec postgres psql -U ovdas -d ovdas -c '\dn'
docker compose exec postgres psql -U ovdas -d ovdas -c '\dt core.*'
docker compose exec postgres psql -U ovdas -d ovdas -c '\dt deteccion
.*'
```

NOTA

Los archivos de migración en `db/` **no** se ejecutan automáticamente. Solo `init.sql` se ejecuta al primer arranque del contenedor. Las migraciones deben aplicarse manualmente según sea necesario.

7. Ejecución y Flujo de Trabajo

7.1 Prueba de Extremo a Extremo (test-flow.sh)

El script `test-flow.sh` ejecuta una batería completa de pruebas automáticas contra el stack en ejecución:

```
# Contra Docker Compose (localhost)
bash scripts/test-flow.sh

# Contra K3s (NodePort)
bash scripts/test-flow.sh k3s

# El script verifica:
# 1. Health check del Core
# 2. Health check del API Gateway
# 3. Ingesta de traza de prueba (POST /ingesta/traza)
# 4. Consulta de estado del evento (GET /evento/{id})
# 5. Listado de eventos (GET /eventos)
# 6. Reporte agregado (GET /reporte/evento/{id})
# 7. Listado de volcanes (GET /volcanes)
```

7.2 Ingesta Manual vía API REST

```
# Ingesta de una sola traza (estacion RPN, volcan 3 = Villarrica)
curl -X POST http://localhost:8000/ingesta/traza \
  -H "Content-Type: application/json" \
  -d '{
    "volcan_id": 3,
    "estacion_id": "RPN",
    "componente": "Z",
    "inicio_unix": 1700000000.0,
    "duracion_seg": 60.0,
    "muestra_hz": 100,
    "extra": {
      "archivo": "/shared/data/RPN.Z.2024001.mseed"
    }
  }'
```

```
# Respuesta esperada:
# {"evento_id": "3-20240101-A1B2", "estado": "INGRESADO",
#  "mensaje": "Evento creado y enviado a deteccion"}
```

Ingesta multi-estación (se esperan picks de N estaciones antes de localizar):

```
# Primera estacion (crea el evento con grupo_id)
curl -X POST http://localhost:8000/ingesta/traza \
  -H "Content-Type: application/json" \
  -d '{
    "volcan_id": 3,
    "estacion_id": "RPN",
    "componente": "Z",
    "extra": {
      "archivo": "/shared/data/RPN.Z.mseed",
      "grupo_id": "GRUPO-2024001-001",
```

```

        "n_estaciones": 3
    }
}'

# Segunda y tercera estacion (se suman al mismo evento)
curl -X POST http://localhost:8000/ingesta/traza \
  -H "Content-Type: application/json" \
  -d '{"volcan_id": 3, "estacion_id": "PFT", "componente": "Z",
    "extra": {"archivo": "/shared/data/PFT.Z.mseed",
              "grupo_id": "GRUPO-2024001-001", "n_estaciones": 3}}'

```

7.3 Consulta de Resultados

```

# Estado del evento individual
curl http://localhost:8000/evento/3-20240101-A1B2

# Listado de ultimos 20 eventos
curl http://localhost:8000/eventos?limite=20

# Reporte completo agregado (via API Gateway)
curl http://localhost:8080/reporte/evento/3-20240101-A1B2

# Filtrar eventos por estado
curl "http://localhost:8080/eventos?estado=COMPLETADO&limite=50"

# Listado de volcanes
curl http://localhost:8080/volcanes

# Consulta directa en PostgreSQL
docker compose exec postgres psql -U ovdas -d ovdas -c \
  "SELECT evento_id, estado, created_at
  FROM core.pipeline_eventos
  ORDER BY created_at DESC LIMIT 10;"

```

7.4 Archivos MiniSEED — Volumen Compartido

Los workers acceden a los archivos MiniSEED a través del volumen compartido `seismic-data` montado en `/shared/data/`.

```

# Copiar MiniSEED al volumen compartido
docker compose cp /ruta/local/archivo.mseed core:/shared/data/

# Verificar archivos disponibles via API
curl http://localhost:8001/archivos

# Listar archivos directamente en el contenedor
docker compose exec worker-deteccion ls /shared/data/

```

8. Descripción de Servicios

8.1 Core Orchestrator

Servicio central que implementa el patrón Saga de Orquestación. Mantiene el estado del pipeline mediante una máquina de estados y coordina los workers a través de colas RabbitMQ.

Método	Endpoint	Descripción
POST	/ingesta/traza	Ingesta traza, crea evento, publica en cola detección
GET	/evento/{id}	Consulta estado actual del evento
GET	/eventos?limite=N	Lista los últimos N eventos
GET	/health	Liveness check

Módulos internos:

- `main.py` — FastAPI app, endpoints, startup/shutdown
- `orchestrator.py` — Máquina de estados, publicación RabbitMQ, consumer de callbacks
- `db.py` — CRUD sobre `core.pipeline_eventos`, validación de estaciones
- `models.py` — Modelos Pydantic (`TraceInput`, `EventoResponse`)
- `config.py` — Constantes, URLs, nombres de colas, enums de estado

8.2 Worker Detección (ADOVE Pipeline)

Implementa el pipeline ADOVE completo para detección y clasificación de señales sísmicas volcánicas usando filtros SOS, STA/LTA, red neuronal VGG16 y detección de distribución fuera de dominio (OOD).

```
Pipeline de procesamiento:
1. Lectura MiniSEED (ObsPy)
2. gapreduce -> eliminacion de gaps
3. Filtro D -> clasificacion espectral
4. Filtro D2 -> deteccion + clasificacion secundaria
5. Deteccion STA/LTA (umbral configurable, default=504)
6. Clasificacion VGG16 (CPU): VT / LP / TR / OT
7. Correccion OOD (cleanlab)
8. Calculo SNR, frecuencia dominante, duracion
9. INSERT en deteccion.resultados
10. Callback -> ovdas.callbacks
```

8.3 Worker Picking (Castilla Picker)

Implementa el algoritmo Castilla para detección automática de fases P y S, basado en ratio STA/LTA para P-wave e inflexión en energía espectral acumulada para S-wave.

```
Pipeline de procesamiento:
1. Consulta archivo en wws.descargas (PostgreSQL)
2. Lectura MiniSEED, extraccion ventana 60s (ObsPy)
3. Remuestreo a 100 Hz si es necesario
4. Filtro paso-banda Butterworth (0.9-12 Hz, orden 6)
5. Normalizacion de amplitud
```



```

6. Deteccion t_P (STA/LTA sobre amplitud normalizada)
7. Deteccion t_S (inflexion en energia espectral acumulada)
8. Calculo SNR, frecuencia dominante (FFT), amplitud pico
9. INSERT en picking.resultados
10. Callback -> ovdas.callbacks

```

8.4 Worker Localización (Hyposat)

Calcula el hipocentro del evento sísmico usando el programa Hyposat. Recibe los picks P/S de todas las estaciones y genera la localización 3D (latitud, longitud, profundidad) con parámetros de calidad.

Flujo:

```

1. Recibe mensaje en ovdas.localizacion
2. Consulta picking.resultados para todos los picks del evento
3. Genera archivo de entrada para Hyposat
4. Ejecuta binario hypo/hyposat (subprocess)
5. Parsea salida de Hyposat
6. INSERT en localizacion.resultados
7. Callback -> ovdas.callbacks

```

8.5 WWS Poller

Servicio de descarga de datos sísmicos desde un servidor Winston Wave Server (WWS). Sondea periódicamente el servidor, descarga segmentos MiniSEED y los almacena en el volumen compartido, registrando cada descarga en la tabla `wws.descargas`.

8.6 API Gateway (API Composition)

Agrega resultados de múltiples schemas de base de datos en una sola respuesta JSON cohesiva, implementando el patrón API Composition con consultas paralelas (`ThreadPoolExecutor`).

Método	Endpoint	Descripción
GET	/reporte/evento/{id}	Reporte consolidado: estado + detección + picking
GET	/eventos	Lista paginada con filtro por estado
GET	/volcanes	Catálogo de volcanes monitoreados
GET	/health	Liveness check con verificación de DB

9. Mensajería RabbitMQ

RabbitMQ actúa como bus de mensajes asíncrono entre el orquestador y los workers. Todas las colas son **durables** (`durable=True`) y los mensajes son **persistentes** (`delivery_mode=2`) para garantizar que no se pierdan ante reinicios.

Cola	Dirección	Propósito
<code>ovdas.deteccion</code>	Core → Worker-Det.	Solicitud de análisis ADOVE
<code>ovdas.picking</code>	Core → Worker-Pick.	Solicitud de picking P/S
<code>ovdas.localizacion</code>	Core → Worker-Local.	Solicitud de hipocentro
<code>ovdas.clasificacion</code>	Core → Worker-Clasif.	Clasificación final (futuro)
<code>ovdas.callbacks</code>	Workers → Core	Resultados y notificaciones

Estructura de mensajes

Listing 2: Mensaje de tarea: Core → Worker

```
{
  "evento_id":    "3-20240101-A1B2",
  "estacion_id":  "RPN",
  "volcan_id":    3,
  "componente":   "Z",
  "inicio_unix":  1700000000.0,
  "duracion_seg": 60.0,
  "muestra_hz":   100,
  "archivo":      "/shared/data/RPN.Z.mseed"
}
```

Listing 3: Mensaje de callback: Worker → Core

```
{
  "tipo":         "DETECCION_COMPLETADA",
  "evento_id":    "3-20240101-A1B2",
  "n_eventos":    2,
  "ok":           true,
  "error":        null
}
```

Monitoreo de colas desde CLI

```
# Acceder a la UI de gestion
# Docker: http://localhost:15672
# K3s:    http://IP_NODO:31672
# Credenciales: ovdas / ovdas

# Verificar colas desde CLI
docker compose exec rabbitmq \
  rabbitmqctl list_queues name messages consumers

# Resultado esperado con stack activo:
# ovdas.deteccion      0    1
# ovdas.picking        0    1
```

```
# ovdas.localizacion 0 1
# ovdas.callbacks    0 1
```

10. Base de Datos PostgreSQL — Esquemas

Schema: core

```
-- Eventos del pipeline
core.pipeline_eventos (
    evento_id          VARCHAR PRIMARY KEY,
    estado              VARCHAR(20),          -- estado actual del pipeline
    volcan_id           INTEGER,
    estacion_id         VARCHAR(10),
    componente          CHAR(1),
    inicio_unix         FLOAT,
    duracion_seg        FLOAT,
    muestra_hz           INTEGER,
    traza_metadata      JSONB,                -- n_estaciones, grupo_id,
    picks, etc.         TEXT,
    error_msg           TEXT,
    created_at          TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
    updated_at          TIMESTAMP WITH TIME ZONE DEFAULT NOW()
)

-- Catalogos
core.volcanes          (volcan_id, nombre, latitud, longitud, altura)
core.estaciones        (estacion_id, nombre, volcan_id, latitud, longitud,
                        altitud, calibracion, activa)
```

Schema: deteccion

```
deteccion.resultados (
    id                  SERIAL PRIMARY KEY,
    evento_id           VARCHAR REFERENCES core.pipeline_eventos,
    estacion            VARCHAR(10),
    componente          CHAR(1),
    snr                 FLOAT,
    label_event         VARCHAR(10),          -- VT / LP / TR / OT
    prob_vt             FLOAT,
    prob_lp             FLOAT,
    prob_tr             FLOAT,
    prob_ot             FLOAT,
    inicio              FLOAT,
    fin                 FLOAT,
    largo               FLOAT,
    prom_ruido_fondo    FLOAT,
    created_at          TIMESTAMP WITH TIME ZONE DEFAULT NOW()
)
```

Schema: picking

```
picking.resultados (
    id                  SERIAL PRIMARY KEY,
    evento_id           VARCHAR REFERENCES core.pipeline_eventos,
    estacion            VARCHAR(10),
    componente          CHAR(1),
```

```
t_p          FLOAT,          -- tiempo llegada onda P (unix)
t_s          FLOAT,          -- tiempo llegada onda S (unix)
snr          FLOAT,
amplitud     FLOAT,
polar        FLOAT,
freq_dom     FLOAT,
created_at   TIMESTAMP WITH TIME ZONE DEFAULT NOW()
)
```

Schema: localizacion

```
localizacion.resultados (
  id          SERIAL PRIMARY KEY,
  evento_id   VARCHAR REFERENCES core.pipeline_eventos,
  tiempo_origen FLOAT,
  lat         FLOAT,
  lon         FLOAT,
  z_m         FLOAT,          -- profundidad en metros
  rmse        FLOAT,
  gap         FLOAT,
  ml          FLOAT,          -- magnitud local
  n_fases     INTEGER,
  n_picks_usados INTEGER,
  created_at   TIMESTAMP WITH TIME ZONE DEFAULT NOW()
)
```

11. Monitoreo y Observabilidad

Logs en tiempo real

```
# Todos los servicios
docker compose logs -f

# Servicio específico
docker compose logs -f core
docker compose logs -f worker-deteccion

# Últimas 100 líneas
docker compose logs --tail=100 worker-deteccion

# En K3s
kubectl -n ovdas logs -f deployment/core
kubectl -n ovdas logs -f deployment/worker-deteccion --all-containers
```

Consultas de monitoreo en PostgreSQL

```
-- Conteo por estado
SELECT estado, COUNT(*)
FROM core.pipeline_eventos
GROUP BY estado ORDER BY COUNT(*) DESC;

-- Eventos con error
SELECT evento_id, error_msg, created_at
FROM core.pipeline_eventos WHERE estado = 'ERROR'
ORDER BY created_at DESC LIMIT 20;

-- Tasa de procesamiento (últimas 24h)
SELECT DATE_TRUNC('hour', created_at) AS hora, COUNT(*) AS eventos
FROM core.pipeline_eventos
WHERE created_at > NOW() - INTERVAL '24 hours'
GROUP BY hora ORDER BY hora;

-- Resultados de detección recientes
SELECT d.evento_id, d.label_event, d.snr, d.prob_vt, d.prob_lp
FROM deteccion.resultadosd
ORDER BY d.created_at DESC LIMIT 10;
```

Métricas de RabbitMQ

```
# Longitud de colas (mensajes pendientes)
docker compose exec rabbitmq \
  rabbitmqctl list_queues name messages messages_ready
  messages_unacknowledged

# Consumers activos por cola
docker compose exec rabbitmq \
  rabbitmqctl list_consumers queue_name channel_details consumer_tag
```

12. Escalado y Alta Disponibilidad (K3s)

El stack Kubernetes incluye un **HorizontalPodAutoscaler (HPA)** para el worker-detección que escala automáticamente entre 1 y 5 réplicas según utilización de CPU (umbral: 70 %).

```
# Ver estado actual del HPA
kubectl -n ovdas get hpa

# Ejemplo de salida:
# NAME                                REFERENCE                      TARGETS  MINPODS
#   MAXPODS REPLICAS                  45%/70%   1      5
# worker-deteccion Deployment/worker-deteccion
#                               2

# Escalar manualmente (override del HPA)
kubectl -n ovdas scale deployment/worker-deteccion --replicas=3

# Escalar worker-picking manualmente
kubectl -n ovdas scale deployment/worker-picking --replicas=2
```

Recursos por servicio (K3s)

Servicio	CPU Req.	CPU Limit	RAM Req.	RAM Limit
core	100m	500m	128Mi	256Mi
worker-deteccion	200m	2000m	128Mi	512Mi
worker-picking	100m	1000m	128Mi	256Mi
worker-localizacion	100m	500m	64Mi	128Mi
wws-poller	50m	200m	64Mi	128Mi
api-gateway	100m	500m	128Mi	256Mi
postgres	200m	1000m	256Mi	1Gi
rabbitmq	200m	1000m	256Mi	512Mi

CONSEJO

El worker-detección es el servicio más intensivo en CPU (VGG16). Para mejorar el rendimiento, considere aumentar el límite de CPU a 4000m en servidores con 8+ cores y ajustar el umbral del HPA al 60 %.

13. Resolución de Problemas

El worker-detección nunca llega a estado “listo”

El modelo VGG16 tarda 60–90 segundos en cargar. Revisar logs:

```
docker compose logs -f worker-deteccion
# Buscar: "Modelos cargados correctamente"
# Si hay error de torch: docker compose build worker-deteccion
```

Error: “estación no encontrada en catálogo”

La estacion_id enviada no existe en core.estaciones. Verificar:

```
SELECT estacion_id, nombre FROM core.estaciones;
-- Si es necesario, ejecutar: migrate_estaciones_completo.sql
```

Eventos bloqueados en estado DETECTANDO

1. Verificar que worker-deteccion está corriendo: `docker compose ps`
2. Verificar mensajes en la cola: `rabbitmqctl list_queues`
3. Revisar logs del worker por errores de lectura del .mseed
4. Confirmar que el archivo existe en `/shared/data/`

PostgreSQL no acepta conexiones

Esperar a que el healthcheck de postgres termine (`pg_isready`). Si el volumen está corrupto:

```
docker compose down -v && docker compose up --build
```

! ADVERTENCIA

`down -v` elimina **permanentemente** todos los datos de la base de datos. Realice un `pg_dump` antes si necesita conservar los datos.

RabbitMQ reporta “connection reset”

Los workers usan `heartbeat=0` para procesos CPU-intensivos. Si el broker reinicia, los workers se reconectan automáticamente (`MAX_RETRY_MQ=10`, `RETRY_DELAY=3s`). Verificar logs del worker.

K3s: imágenes no encontradas (`ErrImageNeverPull`)

Las imágenes deben importarse explícitamente a containerd de K3s:

```
bash scripts/build-images.sh
sudo k3s ctr images ls | grep ovdas
```

NodePort no accesible desde red externa (Fedora/RHEL)

```
firewall-cmd --add-port=30800/tcp --permanent
```



```
firewall-cmd --add-port=30808/tcp --permanent  
firewall-cmd --add-port=31672/tcp --permanent  
firewall-cmd --reload
```

14. Módulos Futuros

Los siguientes módulos están planificados para fases posteriores:

Módulo	Cola	Descripción	Estado
worker-parametros	ovdas.parametros	RSAM, SSAM, energía sísmica, DRS	Pendiente
worker-alertas	ovdas.alertas	Motor de alertas por umbrales	Pendiente
dashboard-frontend	HTTP/WebSocket	Interfaz web tiempo real (React + D3.js)	Diseño
Redis cache	—	Cache de reportes para API Gateway	Diseño
Prometheus/Grafana	—	Métricas y dashboards operacionales	Pendiente

Patrón para integrar un nuevo worker

1. Crear directorio worker-`<nombre>/`
 Dockerfile
 requirements.txt
 worker.py
2. En worker.py:
 - a. Conectar a RabbitMQ (`pika`, `basic_consume` en `ovdas.<nombre>`)
 - b. Procesar mensaje
 - c. INSERT en schema propio
 - d. Publicar callback en `ovdas.callbacks`
 con tipo `<NOMBRE>_COMPLETADO`
3. En core/config.py:
 - Agregar `QUEUE_<NOMBRE> = 'ovdas.<nombre>'`
 - Agregar `TipoCallback.<NOMBRE>_COMPLETADO`
 - Agregar `Estado.<NOMBRE>ING` si aplica
4. En core/orchestrator.py:
 - Agregar handler `_manejar_<nombre>_completado()`
 - Agregar transicion en `_procesar_callback()`
5. En db/init.sql o nueva migracion:
 - Crear schema `<nombre>.resultados`
6. En docker-compose.yml y k8s/:
 - Agregar servicio y manifiestos correspondientes

15. Referencias

1. Arquitectura de Microservicios

Richardson, C. (2018). *Microservices Patterns*. Manning Publications.

2. Saga Pattern

Garcia-Molina, H., & Salem, K. (1987). Sagas. *ACM SIGMOD Record*, 16(3), 249–259.

3. ADOVE / OVDAS-Core

Aracena, J. (2025). *Rediseño Arquitectónico del Sistema OVDAS*. Tesis de pregrado, Universidad de La Frontera (UFRO), Temuco, Chile.

4. ObsPy

Beyreuther, M., et al. (2010). ObsPy: A Python Toolbox for Seismology. *Seismological Research Letters*, 81(3), 530–533.

5. RabbitMQ

RabbitMQ Documentation. <https://www.rabbitmq.com/documentation.html>

6. K3s — Lightweight Kubernetes

Rancher Labs. <https://k3s.io/>

7. FastAPI

Ramírez, S. *FastAPI Documentation*. <https://fastapi.tiangolo.com/>

8. Hyposat

Schweitzer, J. (2001). HYPOSAT — An Enhanced Routine to Locate Seismic Events. *Pure and Applied Geophysics*, 158(1–2), 277–289.

Manual generado para OVDAS-Core v2.3.0 · OVDAS – UFRO 2026 · Confidencial – Uso interno